

Aufgabenblatt: Eingabe, Wrapper, Casting & Parsing

Aufgabe 1: Persönliche Begrüßung

Beschreibung: Der `Scanner` liest Tastatureingaben ein und stellt fuer jeden Datentyp eine eigene Methode bereit: `next()` fuer einen String-Token, `nextInt()` fuer eine ganze Zahl, `nextDouble()` fuer eine Kommazahl. Das Objekt wird einmalig angelegt und fuer alle Eingaben wiederverwendet.

Aufgabenstellung: Lese mit dem `Scanner` den Namen und das Alter des Benutzers ein und gib anschliessend eine personalisierte Begrüßung aus.

```
Wie heisst du? Max
Wie alt bist du? 32
Hallo Max, du bist 32 Jahre alt!
```

Aufgabe 2: Warenkorb

Beschreibung: Eingaben verschiedener Typen muessen in einem Programm zusammenspielen. Ein Artikelname ist ein String, ein Preis eine Kommazahl, eine Stueckzahl eine ganze Zahl. Erst durch die Kombination ergibt sich ein sinnvolles Ergebnis.

Aufgabenstellung: Lies Artikelname, Preis und Stueckzahl mit dem `Scanner` ein. Gib die bestellte Menge und den berechneten Gesamtpreis mit zwei Nachkommastellen aus.

```
Welchen Artikel moechtest du kaufen? Pizza
Was kostet der Artikel? 14,99
Wie viele Stueck moechtest du kaufen? 8
```

```
Du hast 8x Pizza gekauft.
Der Gesamtpreis ist 119,92 EUR
```

Aufgabe 3: Notendurchschnitt

Beschreibung: Mehrere Eingaben desselben Typs werden nacheinander eingelesen, gemeinsam verarbeitet und das Ergebnis formatiert ausgegeben. `printf` mit `%.2f` rundet das Ergebnis auf exakt zwei Nachkommastellen.

Aufgabenstellung: Lies drei Noten als Kommazahlen ein, berechne den Durchschnitt und gib ihn auf zwei Nachkommastellen gerundet aus.

```
Gib drei Noten ein:
Note 1: 1,5
Note 2: 2,0
Note 3: 3,0
Der Durchschnitt ist: 2,17
```

Aufgabe 4: Parsing in der Praxis

Beschreibung: Parsing ist das Gegenstueck zum Casting: Statt Zahlen in andere Zahlentypen umzuwandeln, wird ein String-Wert in einen echten Datentyp ueberfuehrt. Das ist unverzichtbar, wenn Daten aus Dateien, Formularen oder Netzwerkantworten als reiner Text ankommen und erst berechenbar gemacht werden muessen.

Aufgabenstellung: Gegeben sind folgende String-Variablen direkt im Code:

```
String textAlter    = "28";
String textGroesse  = "1.75";
String textAktiv    = "true";
```

Parse alle drei Werte mit den passenden Methoden (`Integer.parseInt` , `Double.parseDouble` , `Boolean.parseBoolean`) in den jeweils korrekten primitiven Datentyp. Berechne aus dem Alter das Geburtsjahr (2026 minus Alter) sowie Groesse hoch 2 und gib alle Ergebnisse aus.

```
Alter:      28
Groesse:    1.75
Aktiv:      true
Geburtsjahr: 1998
Groesse^2:  3.0625
```

[Bonus] Bandnamen-Generator

Beschreibung: String-Eingaben lassen sich direkt zu neuen Saetzen zusammensetzen. `scanner.next()` liest dabei jeweils einen einzelnen Token ein, also ein Wort ohne Leerzeichen.

Aufgabenstellung: Lese die Geburtsstadt und das Lieblingstier des Benutzers (jeweils ein Wort) mit dem `Scanner` ein. Kombiniere beide Eingaben zu einem Bandnamen nach dem Schema: Stadtname + "er" und Tiername + "s". Gib das Ergebnis aus.

```
Willkommen beim Bandnamen-Generator!
In welcher Stadt bist du aufgewachsen? Berlin
Was ist dein Lieblingstier (auf Englisch)? Dog
```

Dein moeglicher Bandname ist: Berliner Dogs

[Bonus] BMI-Rechner

Beschreibung: Der Body-Mass-Index ist eine einfache Kennzahl aus Gewicht und Koerpergroesse. Das Programm verbindet zwei Scannereingaben, eine Formel und eine formatierte Ausgabe in einem realistischen Szenario.

Aufgabenstellung: Lies Koerpergewicht in Kilogramm und Koerpergroesse in Metern mit dem `Scanner` ein. Berechne den BMI nach der Formel $BMI = \text{Gewicht} / (\text{Groesse} * \text{Groesse})$ und gib das Ergebnis auf zwei Nachkommastellen aus.

```
Gewicht in kg: 75
Groesse in m: 1,80

Dein BMI betraegt: 23,15
```

[Bonus] Wrapper-Konstanten erkunden

Beschreibung: Jeder primitive Datentyp hat feste Wertegrenzen. Die zugehoerigen Wrapper-Klassen stellen diese Grenzen als Konstanten bereit. Wer die Grenzen kennt, versteht, wann ein Datentyp zu klein wird und warum es dann zu Überlaeufen kommen kann.

Aufgabenstellung: Gib die Minimal- und Maximalwerte fuer `Integer`, `Long` und `Double` mithilfe der entsprechenden Wrapper-Konstanten aus. Berechne ausserdem, wie viele verschiedene `Integer`-Werte moeglich sind und speichere das Ergebnis als `long`.

```
=== Wertebereiche ===
Integer: -2147483648 bis 2147483647
Long:    -9223372036854775808 bis 9223372036854775807
Double:  4.9E-324 bis 1.7976931348623157E308

Anzahl moeglicher Integer-Werte: 4294967296
```